Day 80 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 4 reference

Watch: Transformers, Part 7: Decoder (2), masked self-attention (Lennart Svensson) | Masking the future in self-attention (Herman Kamper)

`day80_decoder_block.py`, segment by segment, in the order `main()` uses things. The attention, encoding, LayerNorm, FFN and data code is copied verbatim from Days 62–78 and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: y = decoder inputs · enc = the encoder's output · n, ne = decoder and encoder lengths · i = query position, j = key position · V = vocabulary size: 2084 (Day 62's 2082 entries, then BOS and EOS)

Segment 1 — The mask and the attentions

causal_mask

MATH

$$mask[i, j] = (j \le i)$$

CODE

```
def causal_mask(n):
    return np.tril(np.ones((n, n), dtype=bool))
```

- `np.tril(A)` — keeps the lower triangle (diagonal included) and sets the rest to 0, here False.
- `dtype=bool` — a boolean mask, ready for Day 75's np.where(mask, S, -np.inf).

attend and attend_backward: three separate inputs

MATH

$$Q = X_q W_q, K = X_k W_k, V = X_v W_v; \quad dX_q = dQ\, W_q^T, \ldots$$

CODE

```
def attend(Xq, Xk, Xv, P, prefix, h, mask):
    Q = split_heads(Xq @ P[prefix + "q"], h)
    K = split_heads(Xk @ P[prefix + "k"], h)
    V = split_heads(Xv @ P[prefix + "v"], h)
    Zh, A = scaled_dot_product_attention(Q, K, V, mask)
    Z = merge_heads(Zh, h)
    return Z @ P[prefix + "o"], dict(Q=Q, K=K, V=V, A=A, Z=Z, Xq=Xq, Xk=Xk, Xv=Xv)
def attend_backward(dO, P, prefix, h, c):
    flat = lambda T: T.reshape(-1, T.shape[-1])
    g = {prefix + "o": flat(c["Z"]).T @ flat(dO)}
    dQ, dK, dV = scaled_dot_product_attention_backward(split_heads(dO @ P[prefix + "o"].T, h),
        c["Q"], c["K"], c["V"], c["A"])
    dQ, dK, dV = merge_heads(dQ, h), merge_heads(dK, h), merge_heads(dV, h)
    g[prefix + "q"], g[prefix + "k"], g[prefix + "v"] = flat(c["Xq"]).T @ flat(dQ),
        flat(c["Xk"]).T @ flat(dK), flat(c["Xv"]).T @ flat(dV)
    return dQ @ P[prefix + "q"].T, dK @ P[prefix + "k"].T, dV @ P[prefix + "v"].T, g
```

- `P[prefix + "q"]` — string concatenation picks the weights: 's' + 'q' for self-attention, 'c' + 'q' for cross-attention.

- `Xq, Xk, Xv` — self-attention passes the decoder three times; cross-attention passes the encoder as Xk and Xv.
- `return dQ @ ..., dK @ ..., dV @ ..., g` — one gradient per input, so the caller adds them where inputs coincide.

DecoderBlock.forward

MATH

$$allowed[i, j] = real[j] \wedge (j \le i); \quad cross: allowed[i, j] = real_{enc}[j]$$

CODE

```
def forward(self, y, pe, ymask, enc=None, emask=None):
    P, h, c = self.P, self.h, {}
    m, n = ymask.shape
    allowed = np.broadcast_to(ymask[:, None, :], (m, n, n))
    if self.causal:
        allowed = allowed & causal_mask(n)[None]
    sm = np.repeat(allowed[:, None], h, axis=1).reshape(m * h, n, n)
    Xqk = y + pe
    a, c["self"] = attend(Xqk, Xqk, y, P, "s", h, sm)
    h1, c["n1"] = layer_norm(y + a, P["g1"], P["b1"])
    if self.cross:
        cm = np.repeat(np.broadcast_to(emask[:, None, None, :], (m, 1, n, emask.shape[1])), h,
            axis=1).reshape(m * h, n, -1)
        ca, c["cross"] = attend(h1, enc, enc, P, "c", h, cm)
        h2, c["n2"] = layer_norm(h1 + ca, P["g2"], P["b2"])
    else:
        h2 = h1
    f, c["ffn"] = ffn(h2, dict(W1=P["W1"], c1=P["c1"], W2=P["W2"], c2=P["c2"]))
    out, c["n3"] = layer_norm(h2 + f, P["g3"], P["b3"])
    self.c = c
    return out
```

- `np.broadcast_to(ymask[:, None, :], (m, n, n))` — a read-only view repeating the key mask for every query row.
- `allowed & causal_mask(n)[None]` — & of two boolean arrays: real AND not in the future.
- `enc=None, emask=None` — default arguments: a decoder-only model calls forward without an encoder.

DecoderBlock.backward

MATH

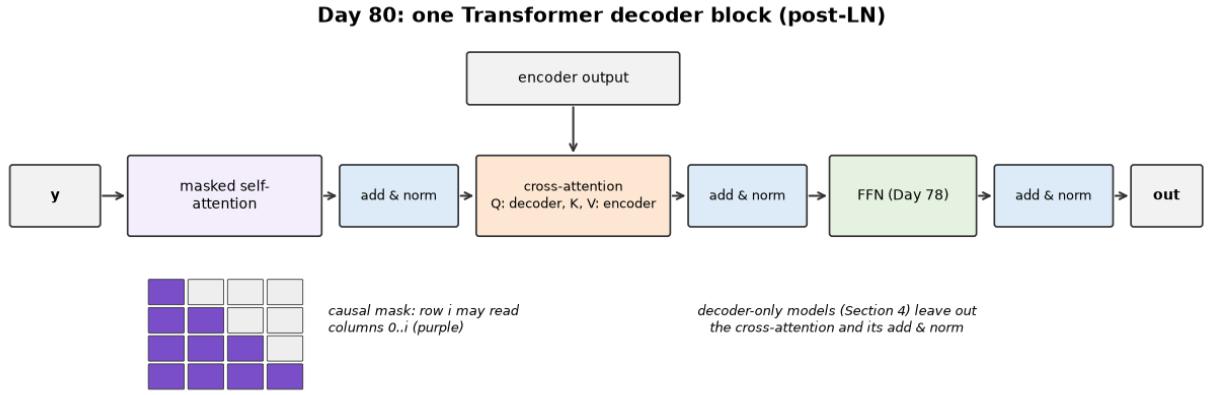
$$dy = ds_1 + dQ\, W_q^T + dK\, W_k^T + dV\, W_v^T$$

CODE

```
def backward(self, dout):
    P, h, c = self.P, self.h, self.c
    g = {}
    ds3, g["g3"], g["b3"] = layer_norm_backward(dout, P["g3"], c["n3"])
    dh2, gf = ffn_backward(ds3, dict(W1=P["W1"], c1=P["c1"], W2=P["W2"], c2=P["c2"]), c["ffn"])
    g.update(gf); dh2 = dh2 + ds3
    denc = None
    if self.cross:
        ds2, g["g2"], g["b2"] = layer_norm_backward(dh2, P["g2"], c["n2"])
        dq, dk, dv, gc = attend_backward(ds2, P, "c", h, c["cross"])
        g.update(gc); dh1 = ds2 + dq; denc = dk + dv
    else:
        dh1 = dh2
    ds1, g["g1"], g["b1"] = layer_norm_backward(dh1, P["g1"], c["n1"])
    dq, dk, dv, gs = attend_backward(ds1, P, "s", h, c["self"])
    g.update(gs)
    return ds1 + dq + dk + dv, denc, g
```

- `dh2 = dh2 + ds3` — the third residual: the gradient also skips the FFN.
- `denc = dk + dv` — the encoder made both the keys and the values of cross-attention.
- `ds1 + dq + dk + dv` — y was the residual AND the queries, keys and values of self-attention: four paths add.

SYSTEM DIAGRAM



One post-LN decoder block, and the causal mask.

Segment 2 — The next-word model

lm_data: inputs and targets shifted by one

MATH

$$input = [BOS, w_1, ..., w_n], \quad target = [w_1, ..., w_n, EOS]: \text{position } i \text{ predicts } w_{i+1}$$

CODE

```
def lm_data(itos, data):
    BOS, EOS = len(itos), len(itos) + 1
    out = []
    for ids, _, mask in data:
        n = mask.sum(axis=1)
        x = np.concatenate([np.full((len(ids), 1), BOS), ids], axis=1)
        y = np.concatenate([ids, np.zeros((len(ids), 1), dtype=int)], axis=1)
        y[np.arange(len(ids)), n] = EOS
        m = np.arange(x.shape[1])[None, :] <= n[:, None]
        x[~m] = PAD
        out.append((x, y, m))
    return out, len(itos) + 2
```

- `np.concatenate([np.full((len(ids), 1), BOS), ids], axis=1)` — a BOS column in front shifts every word one place right.
- `y[np.arange(len(ids)), n] = EOS` — fancy indexing: in row r, column n[r] (just after its last word).
- `x[~m] = PAD` — ~ flips a boolean mask: every position after BOS + n words becomes padding.

NextWord.forward: scores for the last position only

CODE

```
def forward(self, x, mask, last_only=False):
    m, n = x.shape
    self.x = x
    self.y = self.block.forward(self.E[x], self.PE[:n][None], mask)
    out = self.y[:, -1] if last_only else self.y.reshape(m * n, -1)
    logits = out @ self.Wout + self.bout
    self.logp = logits - logits.max(axis=1, keepdims=True)
    self.logp -= np.log(np.exp(self.logp).sum(axis=1, keepdims=True))
    return self.logp
```

- `self.y[:, -1] if last_only else ...` — word-by-word evaluation needs only the last position's 2084 scores, not every position's.
- `**{"b." + k: v for ...}` — in params(): unpacks a dictionary comprehension into keyword arguments of dict().

Segment 3 — Experiments

evaluate: the past-only branch

CODE

```
lens = mask.sum(axis=1)
for L in range(1, lens.max() + 1):
    rows = np.where(lens >= L)[0]
    for s in range(0, len(rows), 256):
        r = rows[s:s + 256]
        xb = x[r, :L]
        lp = net.forward(xb, xb != PAD, last_only=True)
        right += (lp.argmax(axis=-1) == y[r, L - 1]).sum(); nll -= lp[np.arange(len(r)), y[r, L -
            1]].sum(); count += len(r)
return right / count, nll / count
```

- `for L in range(1, lens.max() + 1)` — one batch per prefix length: every sentence long enough, cut to its first L inputs.
- `x[r, :L]` — nothing after position L-1 is given to the model, so even the unmasked model cannot peek.
- `np.where(lens >= L)[0]` — the row numbers where the condition holds.

demo_baselines: a bigram table

CODE

```
pair = np.zeros((V, V), dtype=np.int32)
np.add.at(pair, (x_tr[m_tr], y_tr[m_tr]), 1)
best_next = pair.argmax(axis=1)
best_next[pair.sum(axis=1) == 0] = counts.argmax()
bi = (best_next[x_te[m_te]] == y_te[m_te]).mean()
```

- `np.add.at(pair, (x_tr[m_tr], y_tr[m_tr]), 1)` — adds 1 at every (previous, next) pair, repeats counted.
- `pair.argmax(axis=1)` — for each previous word, the most common next word.
- `best_next[pair.sum(axis=1) == 0] = ...` — boolean indexing on the left of =: words never seen as 'previous' fall back.

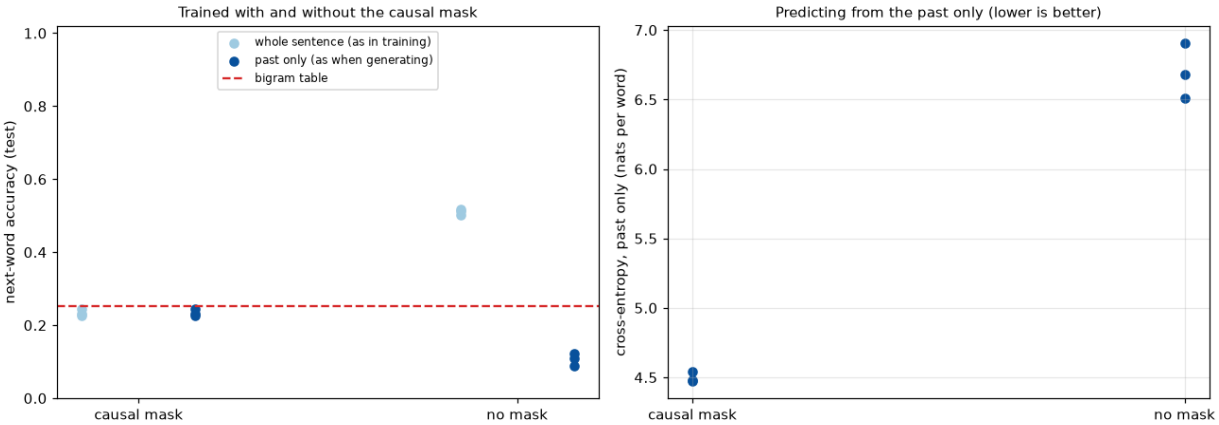
demo_same_answer: one pass against one pass per prefix

CODE

```
worst = 0.0
for L in range(1, T + 1):
    rows = mb[:, L - 1]
    if rows.any():
        part = net.forward(xb[rows, :L], xb[rows, :L] != PAD, last_only=True)
        worst = max(worst, np.abs(part - full[rows, L - 1]).max())
```

- `rows = mb[:, L - 1]` — a boolean array: the sentences that predict something at position L-1.
- `full[rows, L - 1]` — boolean and integer indexing together: those sentences, that position.

REAL GRAPH



Real graph from the script: accuracy both ways, and cross-entropy from the past only.

Quick Syntax Glossary — new or reinforced constructs

- `np.tril(A)` — the lower triangle of a matrix.
- `np.broadcast_to(a, shape)` — a view of a repeated along new axes, without copying.
- `~mask` — boolean NOT.
- `a[b_mask] = v` — assign only where a mask is True.
- `np.add.at(A, (rows, cols), 1)` — count pairs, repeats included.
- `np.where(cond)[0]` — the indices where a 1-D condition is True.
- `def f(x, enc=None)` — an optional argument, None when absent.

End of Day 80 syntax reference. For the experiments and what they showed, see the Day 80 complete guide.